

FinSLM: A Financial News Small Language Model Trained from Scratch

Mahadev Balla^{a,1} and Tanay Kinariwala^{a,2}

^aBharatiya Vidya Bhavan's Sardar Patel Institute of Technology (SPIT), Mumbai, India

Abstract—In this work, we train a small GPT-style language model on a corpus of financial news text collected from multiple sources. The final dataset contains approximately 480 million training tokens. The model consists of a 6-layer, 6-head, 384-dimensional decoder-only transformer with a context length of 128 tokens (approximately 30M parameters). Two training configurations were evaluated: one using a smaller effective batch size with a higher learning rate, and another using a larger batch size with a lower learning rate. The first configuration achieved better performance, reaching a validation loss of 3.508 (perplexity ≈ 33.4), while the second converged more slowly and reached a higher final loss. Generated outputs show that the model captures domain-specific vocabulary and structure, but struggles with factual consistency and numerical accuracy. Overall, the results highlight the impact of training configuration on convergence and suggest that small models can learn useful domain-specific patterns from scratch.

Keywords—Language Modelling, Financial Text, GPT, Small Language Model, Domain-Specific Training

1. INTRODUCTION AND SETUP

Financial news is a reasonably structured domain for training a small language model from scratch. Articles such as earnings reports, central bank announcements, merger news, and market commentary tend to follow similar formats and use consistent vocabulary. Because of this, even a small model has a reasonable chance of picking up domain-specific patterns, compared to open-domain text where the variety is much higher.

Our goals were: (1) replace the course notebook's TinyStories corpus with a financial news dataset, (2) make training resilient to T4 session timeouts via Drive checkpointing, and (3) empirically compare two gradient accumulation strategies to understand their effect on convergence.

Hardware Constraints

All experiments were run on the Google Colab free tier (NVIDIA Tesla T4, 16 GB VRAM). Since the T4 does not support bfloat16, training was done using float16 with gradient scaling. Because of session time limits, longer runs required checkpointing to Google Drive so that training could resume after disconnections.

2. DATASET CONSTRUCTION (Q1)

2.1. Source Datasets

We combined three publicly available HuggingFace datasets:

- **Bloomberg Financial News** [8] — wire-style articles; the largest source with $\approx 447k$ articles.
- **Financial News Articles** [7] — international financial news; $\approx 306k$ articles.
- **Indian Financial News** [9] — India-specific market and corporate coverage; $\approx 27k$ articles.

Each dataset used different column names for the article text (e.g., `text`, `Article`, `Content`), so all sources were first normalised into a common `text` field before merging. Articles shorter than 100 characters were removed to filter out very short or incomplete entries.

This report describes the implementation and training of a small GPT-style language model on financial news data using the nanoGPT framework [6]. Two training configurations were compared under the constraints of a Tesla T4 GPU on Google Colab, with checkpointing used to handle session interruptions.

Table 1. Dataset Composition After Filtering

Source	Articles	Share
Bloomberg Financial News	446,762	57.3%
Financial News Articles	306,242	39.3%
Indian Financial News	26,108	3.4%
Train split (98%)	760,006	
Validation split (2%)	15,511	

Minimum length filter: 100 characters. Random split seed: 42.

On average, articles contained around 423 words (median: 299), indicating a mix of shorter updates and longer reports.

2.2. Tokenisation

We used the GPT-2 Byte-Pair Encoding (BPE) tokenizer [2] through `tiktoken` (vocabulary size: 50,257). Articles were tokenised using `encode_ordinary`, concatenated, and written to binary `.memmap` files. This allows efficient random access during training without loading the full dataset into memory.

The tokenised files were also backed up to Google Drive so that tokenisation did not have to be repeated after session restarts.

- **Training tokens:** ≈ 480 million
- **Validation tokens:** ≈ 10 million
- **Context length:** 128 tokens

Challenges (Q1).

- A few practical issues came up during dataset preparation.
- (1) *Schema differences:* each dataset used a different format, so preprocessing was needed to standardise the text field.
 - (2) *Source imbalance:* Bloomberg makes up about 57% of the data, which tends to bias the style and entities seen in generated outputs (discussed later in Section 7).
 - (3) *Noise in smaller sources:* the Indian Financial News dataset occasionally contains inconsistent formatting and mixed-language text, which introduces some noise into the overall corpus.

3. MODEL ARCHITECTURE

The model follows a standard decoder-only GPT architecture [4, 5], based on the nanoGPT-style implementation used in the course notebook, without major structural changes.

It consists of 6 transformer blocks. Each block applies pre-LayerNorm followed by causal self-attention with 6 heads (head dimension 64), and then a position-wise feed-forward network with hidden size $4 \times 384 = 1,536$ using GELU activation. Residual connections are used around both the attention and feed-forward components.

Token embeddings and learned positional embeddings (both 384-dimensional) are added and passed through the stack of transformer blocks. The final representations are normalised and projected to vocabulary logits using a linear layer with weight tying.

Table 2. Model Architecture

Parameter	Value
Transformer layers (n_{layer})	6
Attention heads (n_{head})	6
Embedding dim (n_{embd})	384
FFN hidden dim	1,536 ($4 \times n_{\text{embd}}$)
Context length	128 tokens
Dropout	0.1
Vocabulary size	50,257
Total parameters	≈ 30 M

The implementation is based on a lightly modified version of nanoGPT. It includes pre-LayerNorm blocks, optional use of PyTorch’s scaled dot-product attention when available, and weight tying between the embedding and output layers. These choices are standard for models of this size and help keep the implementation efficient while remaining close to the original GPT design.

4. TRAINING PIPELINE MODIFICATIONS (Q3)

A few changes were made to the base notebook training loop, mainly due to hardware constraints and to better suit the dataset.

Float16 and GradScaler. The T4 GPU does not support bfloat16, so training was done in float16. To avoid numerical instability, `torch.amp.GradScaler` was used for loss scaling [3].

Learning rate schedule. Instead of using a constant learning rate, we used linear warmup followed by cosine decay (`SequentialLR`) [1]. Warmup helps stabilise the initial phase of training, while cosine decay gradually reduces the learning rate as training progresses.

Gradient accumulation. Gradients were accumulated over N steps before each optimiser update to increase the effective batch size without exceeding VRAM limits. We compared $N = 4$ (Run 1) and $N = 32$ (Run 2).

One implementation detail was important here: when using float16, `scaler.unscale_(optimizer)` must be called before gradient clipping. This step was missing in Run 1, which caused all recorded gradient norms to appear as 0.0. This was fixed in Run 2.

Drive-backed checkpointing. To handle Colab session interruptions, the full training state (model, optimiser, scheduler, GradScaler, iteration count, and loss history) was saved to Google Drive every 500 iterations. Training could then resume from the latest checkpoint.

Early stopping. Training was stopped if validation loss did not improve for 8 consecutive evaluation intervals (i.e., 4,000 iterations).

Table 3. Training Hyperparameters: Run 1 vs. Run 2

Parameter	Run 1	Run 2
Learning rate	3×10^{-4}	1×10^{-4}
Min LR	3×10^{-5}	1×10^{-5}
Warmup steps	500	1,000
Max iterations	20,000	60,000
Grad. accum. (N)	4	32
Eff. batch (tokens)	16,384	131,072
Grad. clip norm	1.0	0.5
Eval / ckpt interval	500 iters	500 iters
Best val. loss	3.508	3.997
Best perplexity	≈ 33.4	≈ 54.5

Eff. batch (tokens) = batch_size $\times$ $N \times$ block_size = $32 \times N \times 128$.

5. TRAINING RUN 1: RESULTS AND ANALYSIS

Run 1 used $\text{lr} = 3 \times 10^{-4}$ and $N = 4$, resulting in 5,000 optimiser updates over 20,000 iterations.

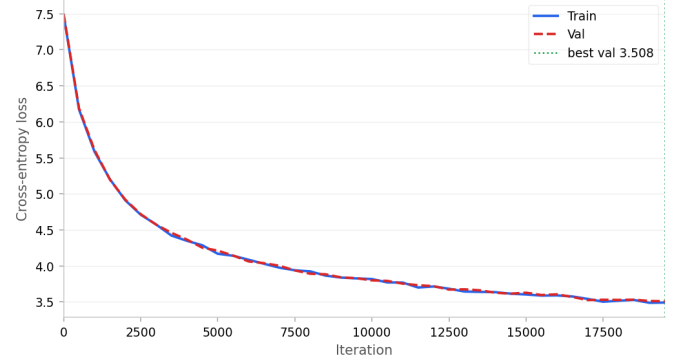


Figure 1. Run 1: training and validation cross-entropy loss over 20,000 iterations (evaluated every 500 steps).

As shown in Fig. 1, the validation loss decreases consistently at every evaluation step, from 7.499 at iteration 0 to 3.508 at iteration 19,500. The gap between training and validation loss at the end is very small (3.489 vs. 3.508), suggesting minimal overfitting.

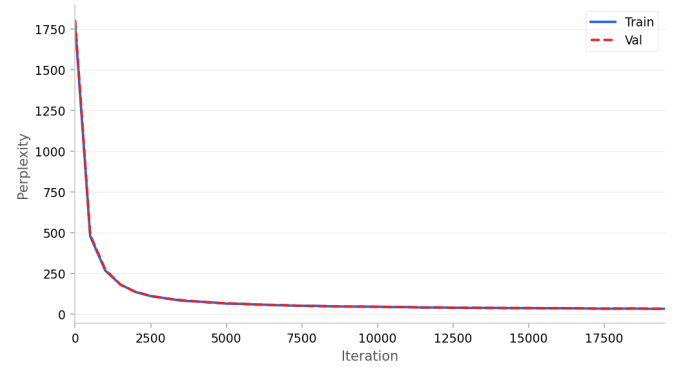


Figure 2. Run 1: training and validation perplexity over 20,000 iterations. Perplexity = e^{loss} .

The same trend is visible in Fig. 2 when viewed in terms of perplexity. The model starts with very high uncertainty (perplexity $\approx 1,800$ at iteration 0) and improves rapidly in the early phase, dropping below 140 within the first few thousand iterations. By the end, it reaches a perplexity of about 33.4.

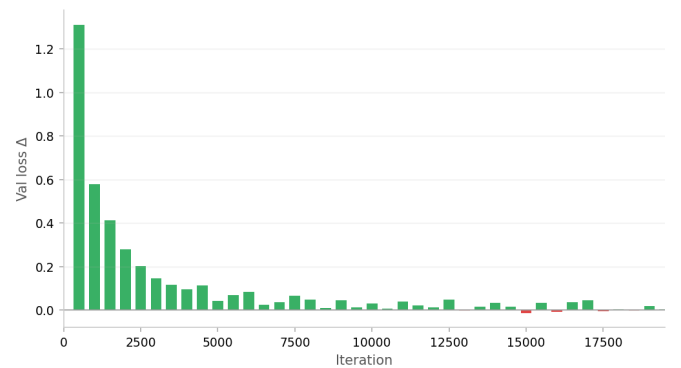


Figure 3. Run 1: validation loss improvement (Δ) at each evaluation.

Fig. 3 shows the change in validation loss at each evaluation step. All values are positive, meaning the model improved at every check-

point without any stagnation or regression. The improvements become smaller over time, which suggests the model is gradually approaching convergence, but it had not clearly plateaued by 20,000 iterations. This was one of the reasons for extending the training in Run 2.

6. TRAINING RUN 2: EXTENDED CONFIGURATION

Run 2 used a larger gradient accumulation factor ($N = 32$) and a lower learning rate (1×10^{-4}). The idea was to test whether a larger effective batch size would lead to more stable training and better final performance. The training budget was extended in stages (20k $\rightarrow$ 40k $\rightarrow$ 60k iterations), resuming each time from the latest checkpoint.

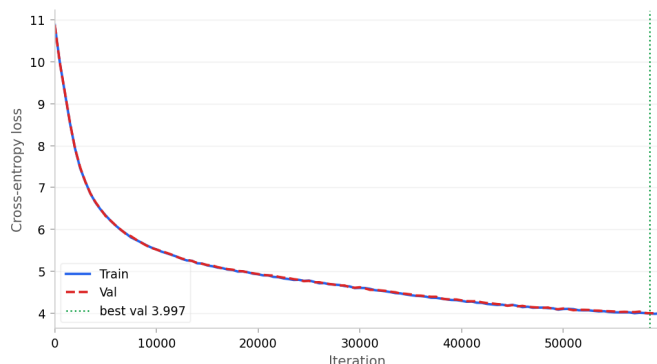


Figure 4. Run 2: training and validation loss over 60,000 iterations.

From Fig. 4, it is clear that convergence is slower compared to Run 1. At iteration 1,000, validation loss is 9.27 (vs. 5.61 in Run 1). Even after 60,000 iterations, the best validation loss reaches only 3.997 (around iteration 57,000), which is still worse than Run 1 by about 0.49.

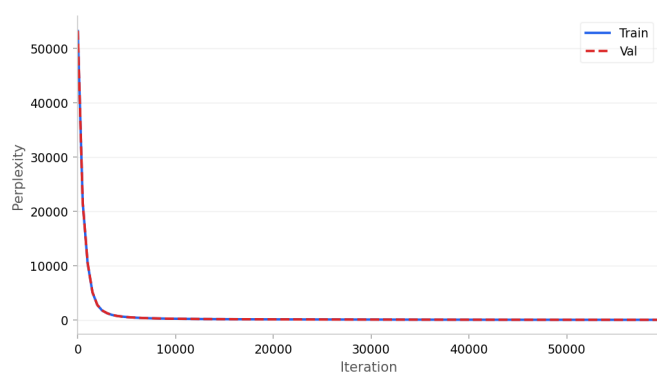


Figure 5. Run 2: perplexity over 60,000 iterations.

This difference is also visible in Fig. 5. Run 2 reaches a best perplexity of about 54.5, compared to 33.4 for Run 1. In practice, this gap shows up in the generated text, where Run 2 outputs tend to be less consistent and more prone to errors (discussed later in Section 7).

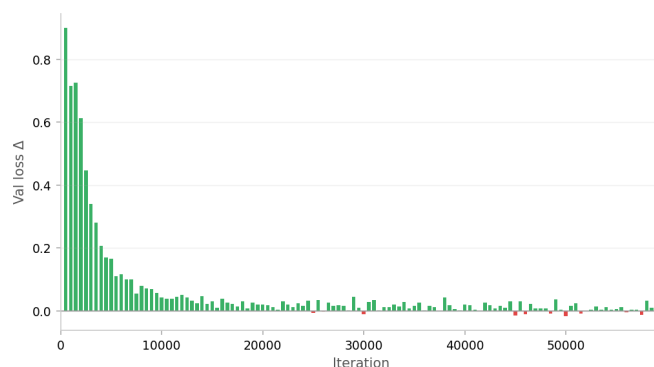


Figure 6. Run 2: validation loss improvement (Δ) per evaluation.

The behaviour is clearer in Fig. 6. Unlike Run 1 (which improved at every step), Run 2 starts showing occasional negative Δ values after roughly 30,000 iterations. This means validation loss sometimes increases between checkpoints, indicating unstable or diminishing improvements. The magnitude of changes also decreases over time, suggesting the model is approaching a plateau.

6.1. Learning Rate Schedule

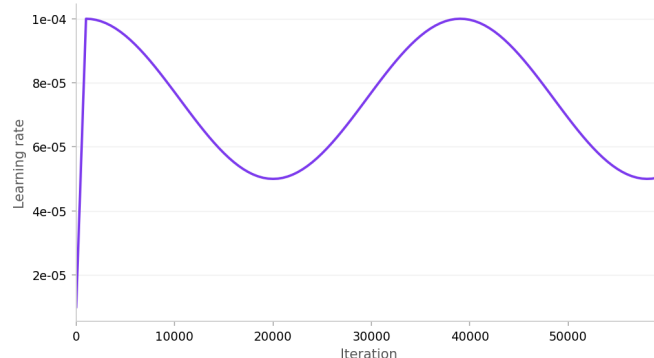


Figure 7. Run 2 learning rate schedule across 60,000 iterations.

Fig. 7 shows that the learning rate schedule was unintentionally reset when training was resumed with a higher `max_iters`. This caused multiple cosine cycles instead of a single smooth decay. These restarts align with the points where validation loss temporarily worsens in Fig. 6, likely due to the optimiser taking larger steps after each reset.

6.2. Gradient Norm

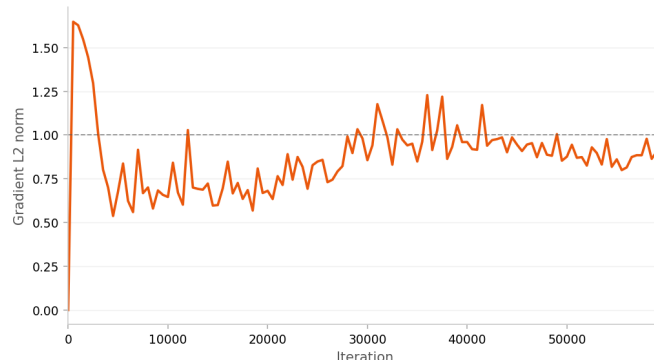


Figure 8. Run 2 pre-clip gradient L2 norm per evaluation checkpoint.

Fig. 8 shows the gradient L2 norm before clipping. After an initial spike during warmup, values mostly lie between 0.5 and 1.2. Since the clipping threshold is set to 0.5, gradients are clipped at almost every step.

This likely limits the benefit of using a larger effective batch size. Although gradient accumulation provides more stable estimates, the clipped updates reduce their impact. In comparison, Run 1 used a higher clipping threshold (1.0) and achieved better results. Adjusting this threshold based on observed gradient norms could improve performance in future runs.

7. QUALITATIVE EVALUATION (Q2)

We generated outputs from both runs (temperature = 0.8, top-k = 50) using a set of domain-specific prompts. Table 4 gives a brief comparison before looking at individual examples.

Table 4. Qualitative Comparison at Best Checkpoints

Aspect	Run 1 (PPL 33.4)	Run 2 (PPL 54.5)
Grammar	Mostly correct	Some errors
Domain vocab	Strong	Moderate
Entity attribution	Plausible but biased	Less consistent
Number formatting	Correct style	Mostly correct
Factual accuracy	Low (hallucinations)	Low (hallucinations)

7.1. Run 1 Outputs (PPL $\approx$ 33.4)

Prompt: “The Federal Reserve raised interest rates by”

...25 basis points since August 2011 to spur growth and boost the U.S. currency. European Central Bank President Mario Draghi said yesterday the ECB had its benchmark interest rate at a record low of zero to 0.3 percent at 3.75 percent...

The model generates a plausible continuation with realistic terminology like “basis points” and correctly references a real ECB president. However, the numerical statements are internally inconsistent. Overall, the structure and style resemble financial reporting, even if the details are not always correct.

Prompt: “Reliance Industries reported a quarterly profit of”

...HK\$7.5 billion on a net loss of HK\$7.3 billion... The Hang Seng Index gained 0.3 percent to 16,972.83. The Shanghai Composite Index slipped 0.9 percent...

Here, the model uses Hong Kong Dollars for an Indian company, which is incorrect. This likely reflects the dominance of Bloomberg-style data in the training set. The sentence structure and formatting are accurate, but the entity-attribute association is not.

Prompt: “The Sensex fell sharply after”

... the greenback weakened 0.5 percent against the U.S. dollar as a global trade war eased... It was down 1.5 percent at 2,973.50 as of the 4 p.m. close of trading...

The model produces a reasonable causal explanation linking currency movements and market behaviour. However, the index value is unrealistic, indicating that while the model captures patterns in language, it does not learn accurate numerical grounding.

Prompt: “Goldman Sachs analysts warned that”

... the world’s biggest oil producer is likely to be looking for higher prices over the next five years... “If the market does look good enough to avoid the market, the stocks are going to be seen as a part of the market,” said Cramer.

The first part is coherent, but the quoted statement becomes repetitive and less meaningful. The name “Cramer” appears without clear context, suggesting a hallucinated attribution. The overall sentence format is still consistent with financial news style.

7.2. Run 2 Outputs After 60k Iterations (PPL $\approx$ 54.5)

Prompt: “The Federal Reserve raised interest rates by”

... a record low of \$29.35 trillion through Sept. 17, according to data compiled by Bloomberg. The Bank of America Merrill Lynch U.S. policy-banking Index has dropped 1.9 percent this year to \$14.15 on the Comex...

The continuation contains structurally correct phrases but incorrect quantities (e.g., “\$29.35 trillion” as a rate change). Compared to Run 1, the output is less precise and more prone to inconsistencies.

Prompt: “The merger between”

... Canada and the United States would be required by the FDA, the SEC said. (For the period, a share price of \$50,000 per common stock...)

The model confuses a corporate merger with a country-level context and introduces unrelated entities such as the FDA. This indicates weaker control over context and entity relationships compared to Run 1.

Perplexity and Output Quality

The difference in validation loss (0.489 nats) corresponds to a noticeable gap in perplexity (33.4 vs. 54.5). This is reflected in the generated outputs: Run 1 generally produces more coherent and consistent text, while Run 2 shows more frequent inconsistencies. This suggests that perplexity is a useful indicator of generation quality at this scale.

8. WHY RUN 1 PERFORMS BETTER

One noticeable difference between the two runs is the number of optimiser updates.

With $N = 4$, Run 1 performs 5,000 updates over 20,000 iterations, while Run 2 ($N = 32$) performs significantly fewer updates over the same number of iterations. Even after extending Run 2 to 60,000 iterations, the total number of updates remains lower.

This likely affects convergence speed. More frequent updates allow the model to adjust parameters more often, which may help it reach a better solution within the same compute budget.

Another factor is the choice of learning rate. Run 2 uses a larger effective batch size but a lower learning rate. This combination results in smaller parameter updates, which may further slow down learning.

Overall, these differences help explain why Run 1 converges faster and reaches a lower validation loss, even with a shorter training time.

Observation

Run 1 achieves better performance with fewer iterations and less training time. This suggests that, for this setup, more frequent updates with a moderate batch size were more effective than using a larger batch with a lower learning rate.

9. FUTURE IMPROVEMENTS (Q4)

If more compute or data were available, a few improvements could be explored.

Longer training. Run 1 was still improving at the end of training without clear overfitting. Extending training further would likely reduce validation loss and improve output quality.

Better data balance. The dataset is dominated by Bloomberg-style articles, which affects generated outputs (e.g., incorrect currency usage for Indian companies). Rebalancing the dataset or adding more India-specific financial data could improve this.

Larger model or context length. Increasing model size or context length may help capture longer dependencies and improve coherence across sentences.

Tokenizer improvements. The current tokenizer splits many financial terms (e.g., tickers or date formats) into multiple tokens. A domain-specific tokenizer could make better use of the context window.

CONCLUSION

In this work, we trained a small GPT-style language model on financial news data and evaluated its behaviour across two training configurations.

The model is able to generate text that follows the general structure and vocabulary of financial reporting. However, it does not consistently produce factually accurate outputs, especially for numerical values and entity-specific details. This is expected given the model size and lack of any grounding mechanism.

Across the two runs, the configuration with a smaller gradient accumulation factor ($N = 4$) and higher learning rate performed better, both in terms of validation loss and training time. The larger batch, lower learning rate setup converged more slowly and did not reach the same level of performance even after longer training.

Overall, the results show that even relatively small models can learn domain-specific patterns, but training setup and data distribution have a significant impact on performance.

REFERENCES

- [1] I. Loshchilov and F. Hutter, “SGDR: Stochastic gradient descent with warm restarts”, *arXiv preprint arXiv:1608.03983*, 2016.
- [2] R. Sennrich, B. Haddow, and A. Birch, “Neural machine translation of rare words with subword units”, pp. 1715–1725, 2016.
- [3] P. Micikevicius et al., “Mixed precision training”, *arXiv preprint arXiv:1710.03740*, 2017.
- [4] A. Vaswani et al., “Attention is all you need”, in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [5] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, “Language models are unsupervised multitask learners”, *OpenAI Blog*, vol. 1, no. 8, 2019.
- [6] A. Karpathy, *nanoGPT: The simplest, fastest repository for training/finetuning medium-sized GPTs*, <https://github.com/karpathy/nanoGPT>, Accessed: May 2025, 2022.
- [7] Ashraq, *Financial News Articles*, <https://huggingface.co/datasets/ashraq/financial-news-articles>, Accessed: May 2025, 2023.
- [8] D. Danou, *Bloomberg Financial News*, https://huggingface.co/datasets/danidanou/Bloomberg_Financial_News, Accessed: May 2025, 2023.
- [9] KDave, *Indian Financial News*, https://huggingface.co/datasets/kdave/Indian_Financial_News, Accessed: May 2025, 2024.

APPENDIX: CODE AND OUTPUTS

The complete Colab notebook, training logs, model checkpoints, and sample outputs are available at the following link:

Google Drive:

<https://drive.google.com/drive/folders/1sMmu3VxEZX1pq8S-eSvZzuDZ3vZ8EmPW?usp=sharing>